



College of Computing and Informatics
Department of Computer Science

Course Syllabus

Course Title

Data Structures

Course Code
1501215

Spring Semester, 2022 / 2023

Course Syllabus

Course Title	Data Structures		
Course Code	1501215		
Credit Hours	3		
Pre-requisite(s)	1501211 Programming II		
Co-requisite(s)	None		
Semester	Spring	Year	2022/2023
Instructors Name	Ashraf Elnagar		
Office Location	W5-219		
Tel. No.	X 2402		
Email	ashraf@sharjah.ac.ae		
Lecture Times			
Office Hours	12:30-2:00 SUN/TUE + by appointment		

Course Description (as in the catalogue):

Basics of algorithm design. Linear Structures: Multidimensional arrays and their storage organization, Lists, Stacks and Queues and introduction to the C++ STL library. Introduction to recursion. Nonlinear structures: trees (binary trees, tree traversal algorithms) and Graphs (graph representation, graph algorithms). Elementary sorting and searching methods.

Course Objectives/Goals:

The goals of this course are to:

- Provide a detail introduction to abstract data types.
- Provide the students with a thorough understanding of linear and nonlinear data structures, and their use to solve different problems.
- Use recursion as a technique to solve problems
- Introduce different searching and sorting techniques and their analysis

Course Learning Outcomes:

By the end of successful completion of this course, the student will be able to:

1. Use linked lists as a problem solving technique.
2. Use recursion as a problem solving technique
3. Apply linear structures such as stacks and/or queues to a variety of problems
4. Represent trees and graph, with their standard manipulation algorithms
5. Use different techniques of searching and sorting.
6. Evaluate various alternatives and select appropriate ADTs.

Alignment of Course Student Learning Outcomes to Program Student Learning Outcomes

CS Program SLOs	Course SLOs
Apply knowledge of computing and mathematics appropriate to the program's student outcomes and to the discipline.	1. Use linked lists as a problem solving technique. 2. Use recursion as a problem solving technique 3. Apply linear structures such as stacks and/or queues to a variety of problems 4. Represent trees and graph, with their standard manipulation algorithms 5. Use different techniques of searching and sorting. 6. Evaluate various alternatives and select appropriate ADTs.
Analyze a problem, and identify and define the computing requirements appropriate to its solution.	1. Use linked lists as a problem solving technique. 2. Use recursion as a problem solving technique 3. Apply linear structures such as stacks and/or queues to a variety of problems 4. Represent trees and graph, with their standard manipulation algorithms 5. Use different techniques of searching and sorting. 6. Evaluate various alternatives and select appropriate ADTs.
Design, implement, and evaluate a computer-based system, process, component, or program to meet desired needs.	1. Use linked lists as a problem solving technique. 2. Use recursion as a problem solving technique 3. Apply linear structures such as stacks and/or queues to a variety of problems 4. 5. Use different techniques of searching and sorting. 6. Evaluate various alternatives and select appropriate ADTs.
Function effectively on teams to accomplish a common goal.	NA
Apply professional, ethical, legal, security and social issues and responsibilities.	NA

Communicate effectively with a range of audiences.	NA
Analyze the local and global impact of computing on individuals, organizations, and society.	NA
Recognize the need for and an ability to engage in continuing professional development.	1. 2. 3. 4. 5. 6. Evaluate various alternatives and select appropriate ADTs.
Use current techniques, skills, and tools necessary for computing practice.	1. Use linked lists as a problem solving technique. 2. Use recursion as a problem solving technique 3. 4. 5. Use different techniques of searching and sorting. 6.
Apply mathematical foundations, algorithmic principles, and computer science theory in the modeling and design of computer-based systems in a way that demonstrates comprehension of the tradeoffs involved in design choices.	1. 2. Use recursion as a problem solving technique 3. 4. 5. Use different techniques of searching and sorting. 6. Evaluate various alternatives and select appropriate ADTs.
Apply design and development principles in the construction of software systems of varying complexity.	1. Use linked lists as a problem solving technique. 2. 3. Apply linear structures such as stacks and/or queues to a variety of problems 4. Represent trees and graph, with their standard manipulation algorithms 5. 6. Evaluate various alternatives and select appropriate ADTs.

Weekly Distribution of Course Topics/Contents

Week	Topic	Comments*	Course SLO
1.	Introduction: Object-Oriented Design (OOD) and C++; Algorithm Analysis: The Big-O Notation. Data Abstraction, Classes, and Abstract Data Types		
2.	Pointers and Array-Based Lists: Pointer data types and variables, Dynamic variables (list & table), Pointer arithmetic, Dynamic arrays, Shallow and deep copying, Peculiarities of classes with pointer data members, Processing lists		6
3.	Linked List: Linked Lists: Some Properties, Item Insertion and Deletion, traversal & search, Building a Linked List, Linked List as an ADT. Linked List Types: Unordered Linked Lists, Ordered Linked Lists, Doubly Linked Lists, Linked Lists with Header and Trailer Nodes, and Circular Linked Lists	HW#1	1
4.			
5.			2
6.	Recursion: Recursive Definitions, Direct and Indirect Recursion, Infinite Recursion, Problem Solving Using Recursion, Recursion or Iteration?, Recursion and Backtracking: 8-Queens Puzzle		
7.			
8.	Stack ADT: definition, array implementation of stacks, linked list implementation of stacks, Application of Stacks: Postfix Expressions Calculator, removing Recursion: Nonrecursive Algorithm to Print a Linked List Backward.	HW#2	3, 6
9.			
10.			
11.	Queues ADT: definition, Implementation of Queues as Arrays, linked implementation of queues, Application of Queues: Simulation		
12.	Searching: sequential search, binary search, Lower Bound on Comparison-Based Search Algorithms, Hashing: Hash Functions Examples, Collision Resolution, Open Addressing, Deletion: Open Addressing, Hashing: Implementation Using Quadratic Probing, Chaining, Hashing Analysis	HW#3	5
13.	Sorting: Selection Sort: Array-Based Lists, Analysis: Selection Sort, Insertion Sort: Array-Based Lists, Insertion Sort: Linked List-Based Lists, Shellsort, Lower Bound on Comparison-Based Sort Algorithms, Quicksort: Array-Based Lists, Analysis: Quicksort, Mergesort: Linked List-Based Lists, Analysis: Mergesort, Heapsort: Array-Based Lists		
14.	Trees: Definition, presentation, tree traversal algorithms; Binary search trees (search, insert, and delete); Binary Search Tree: Analysis, Nonrecursive Binary Tree Traversal Algorithms, Binary Tree Traversal and Functions as Parameters, AVL (Height-Balanced) Trees: Insertion, AVL	HW#4	4, 6

15.	Tree Rotations, Deletion from AVL Trees Graphs: Graph Definitions and Notations, Graph Representation, Adjacency Matrices, Adjacency Lists, Operations on Graphs, Graphs as ADTs, Graph Traversals Shortest Path Algorithm, Shortest Path, Minimum Spanning Tree		
16.	STL: General description		6

* In the comments, you can add the relevant chapter or notes, etc.

Information on out-of-class assignments with due dates for submission

Assignment/Activity	Due Date	Comments
HW 1	End of week # 06	
HW 2	End of week # 10	
HW 3	End of week # 12	
HW 4	End of week # 15	

Students' Assessment:

Students are assessed as follows:

Assessment	Date	Weight (%)
Class work (HW)	Weeks 1-15	20
Class work (Quizzes)	Weeks 1-15	10
Midterm	Week #8/#9	25
Final Exam	Week # 16/#17	45
Total		100

** You can modify / add other tools relevant to the course.

Course Outcome Assessment Plan:

Course LOs	Teaching/Learning Method(s)	Assessment Tool(s)	Performance Indicators
Use linked lists as a problem solving technique.	- Lectures - Tutorials	- Quizzes - Exams - Assignments	
Use recursion as a problem solving technique	- Lectures - Tutorials - Problem solving	- Quizzes - Exams - Assignments	
Apply stacks and queues to a variety of problems	- Lectures - Tutorials - Case studies - Seminars	- Quizzes - Exams - Project	
Represent trees and graph, with their standard manipulation algorithms	- Lectures - Tutorials - Projects - Team discussions	- Quizzes - Exams - Project	

Use different techniques of searching and sorting.	- Lectures - Tutorials - Projects - Team discussions	- Quizzes - Exams - Project	
Evaluate various alternatives and select appropriate ADTs.	- Lectures - Tutorials - Case studies	- Quizzes - Exams - Project	

Teaching and Learning Resources:

Text Book(s):

Malik, D. (2012). Data Structures Using C++. 2nd Edition. Thomson, Course Technology.

Recommended Readings:

Weiss, M. (2013). Data Structures and Algorithms Analysis in C++. 4th Edition. Addison Wesley.

Other Resources: Computer Usage

- C++ ompiler
- Open lab for students to solve their programming assignments.

Attendance policy:

Attendance is compulsory. A student missing 10% (6 hours) of the total allocated course hours will receive 1st warning notice and a student missing 15% (9 hours) will receive 2nd warning notice. A student missing 20% (12 hours or more) will be forced to withdraw (in accordance with the university regulations).

Plagiarism/Cheating:

Students are expected to do their own work. You are allowed to work on assignments in teams only if specified by the instructor. In other words, students are encouraged to communicate about general principles of the course, but all assigned homework must be done on an individual basis. The instructor is available to provide any assistance that you may need. Cheating is considered a serious offense by the university. You should be aware of the severe penalty for cheating (refer to the student code of conduct published in the university catalogue).

Practical Aspect of the course:

Students are given 5 to 6 programming assignments covering the topics covered in the course. Each assignment is usually made of a number of problems. Usually one of these problems requires a lot programming. The other problems are designed in a way to give the student the chance to implement to a particular data structures to solve the given problem.

In-class Activities/Teaching Methodology:

Lecturing, Q&A on the board, program implementation using CodeBlocks IDE, team-based work on problems (peer learning).

Teaching and Learning:

The course includes two lectures per 15 weeks, which include quizzes, and assignments. The quizzes and assignments are on the same level of difficulty with the midterm and final exams. The assignments were designed to inforce the problem solving and critical thinking skills.